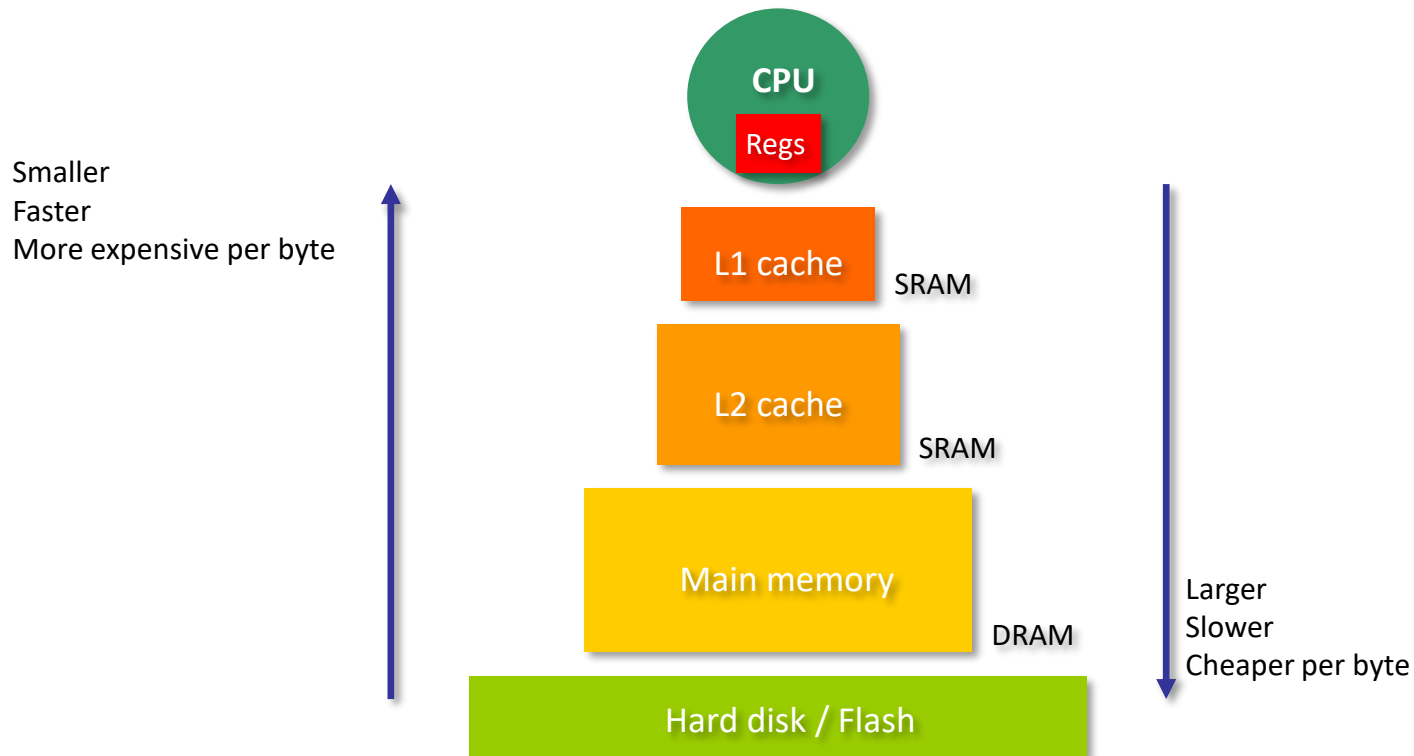


# Memory Hierarchy

## Introduction - §2.1

### (summary of Appendix B)

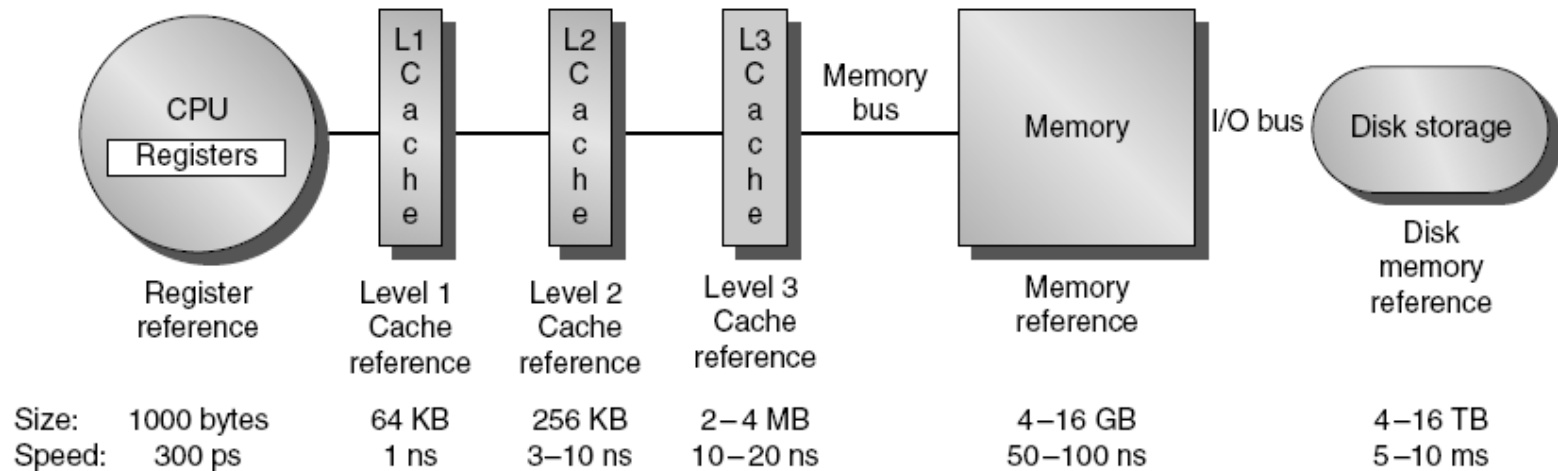


# The Principle of Locality

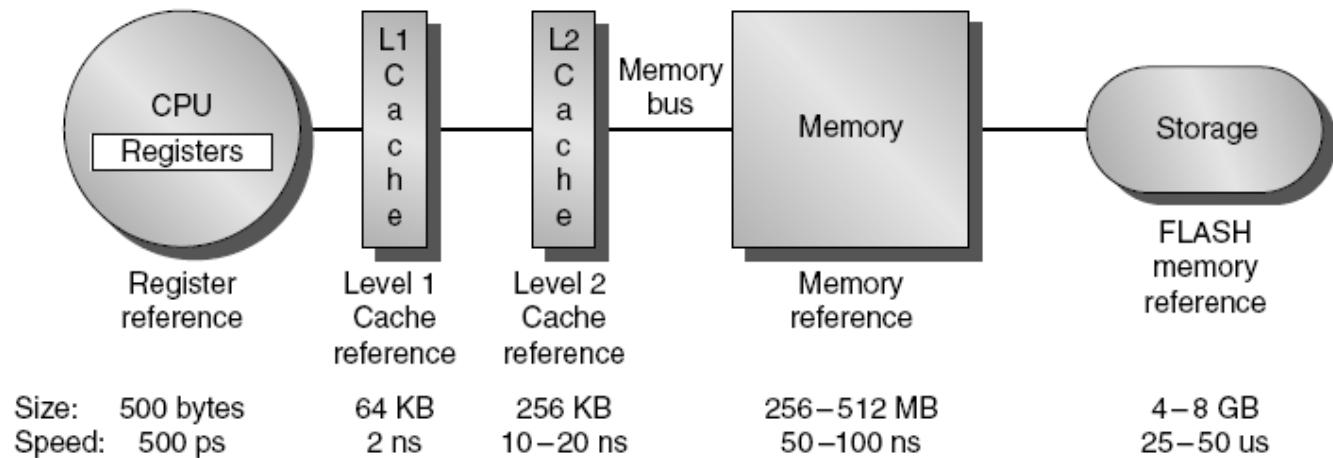


- Two Different Types of Locality:
  - **Temporal Locality** (Locality in Time): If an item is referenced, it will tend to be referenced again soon.
  - **Spatial Locality** (Locality in Space): If an item is referenced, items whose addresses are close by tend to be referenced soon.
- **Memory Hierarchy Terminology:**
  - block: minimum unit of data transferred between levels (e.g., cacheline)
  - hit: data requested is in the cache level
  - miss: data requested is not in the cache level
  - hit rate: percentage of hits (sometimes called hit ratio)
  - miss rate: percentage of miss (sometimes called miss ratio)
  - hit time: the time to satisfy request in case of a hit
  - miss penalty: the time to satisfy a request in case of a miss

# Memory Hierarchy



(a) Memory hierarchy for server



(b) Memory hierarchy for a personal mobile device

# Cache performance



Example: Consider a 5-stage (in order) pipeline and assume that

- With perfect cache,  $\text{CPI} = C$
- 40% of instructions are load/store (access cache for data)
- Miss penalty = 30 clock cycles
- Miss rate = 2% (for each of the I and D caches)

Number of memory accesses per instruction = 1.4

$$\begin{aligned}\text{Actual CPI} &= C + \text{av. memory stalls per instruction due to cache miss} \\ &= C + \text{av. cache misses per instruction} * \text{miss penalty} \\ &= C + (1.4 * 0.02) * 30 = C + 0.84\end{aligned}$$

NOTE: miss penalty of L1 cache is calculated based on the hit time, miss rate and miss penalty of the L2 cache.

## 4 Questions for Memory Hierarchy:



- Q1: Where can a block be placed in the upper level?  
*(Block placement)*
- Q2: How is a block found if it is in the upper level?  
*(Block identification)*
- Q3: Which block should be replaced on a miss?  
*(Block replacement)*
- Q4: What happens on a write?  
*(Write strategy)*

Note: a block (cache line) is the unit of traffic between memory and cache. A block is usually from 4 to 8 words.

# Simplest cache: Direct Mapped

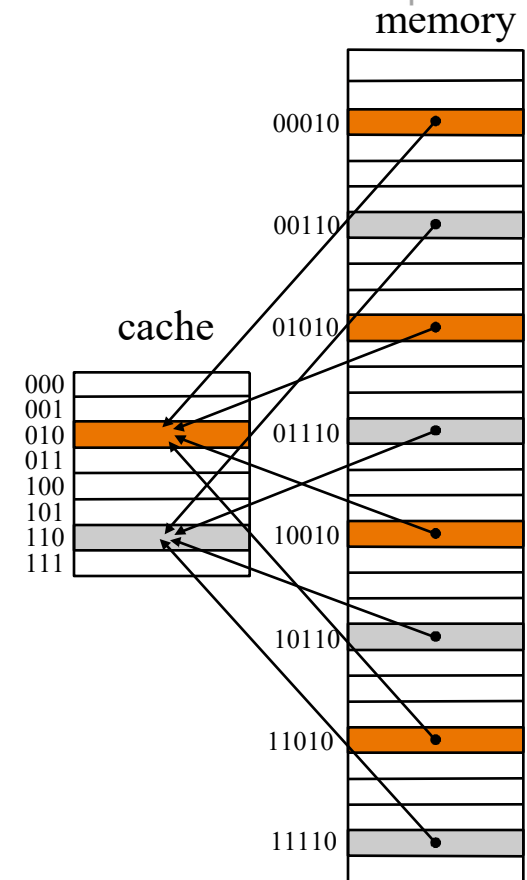
For each item (block) of data in memory, there is exactly one location in the cache where it might be.

$$\text{Index} = (\text{memory address}) \bmod (\text{cache size})$$

Cache location (block address)

**Note:** lots of items in memory share locations in cache

- Cache can be accessed using the low-order address bits
- Need to *tag* data using the high order address bits
- A valid bit is used for each block



# Example

## Cache

Index Valid Tag Data

|    |  |  |  |
|----|--|--|--|
| 00 |  |  |  |
| 01 |  |  |  |
| 10 |  |  |  |
| 11 |  |  |  |

### Q1: Is it there?

Compare the cache tag to the high order 2 memory address bits to tell if the memory block is in the cache

(block address) modulo (cache size)

## Main Memory

One word blocks  
Two low order bits define the byte in the word (32b words)

### Q2: How do we find it?

Use next 2 low order memory address bits – the index – to determine which cache block (i.e., modulo the number of blocks in the cache)

# Direct Mapped Cache

❑ Consider the main memory word reference string

Start with an empty cache - all blocks initially marked as not valid

0 1 2 3 4 3 4 15

**0 miss**

|    |        |
|----|--------|
| 00 | Mem(0) |
|    |        |
|    |        |
|    |        |

**1 miss**

|    |        |
|----|--------|
| 00 | Mem(0) |
| 00 | Mem(1) |
|    |        |
|    |        |

**2 miss**

|    |        |
|----|--------|
| 00 | Mem(0) |
| 00 | Mem(1) |
| 00 | Mem(2) |
|    |        |

**3 miss**

|    |        |
|----|--------|
| 00 | Mem(0) |
| 00 | Mem(1) |
| 00 | Mem(2) |
| 00 | Mem(3) |

**4 miss**

01

|               |                   |
|---------------|-------------------|
| <del>00</del> | <del>Mem(0)</del> |
| 00            | Mem(1)            |
| 00            | Mem(2)            |
| 00            | Mem(3)            |

4

**3 hit**

|    |        |
|----|--------|
| 01 | Mem(4) |
| 00 | Mem(1) |
| 00 | Mem(2) |
| 00 | Mem(3) |

**4 hit**

|    |        |
|----|--------|
| 01 | Mem(4) |
| 00 | Mem(1) |
| 00 | Mem(2) |
| 00 | Mem(3) |

**15 miss**

11

|               |                   |
|---------------|-------------------|
| 01            | Mem(4)            |
| 00            | Mem(1)            |
| 00            | Mem(2)            |
| <del>00</del> | <del>Mem(3)</del> |

15

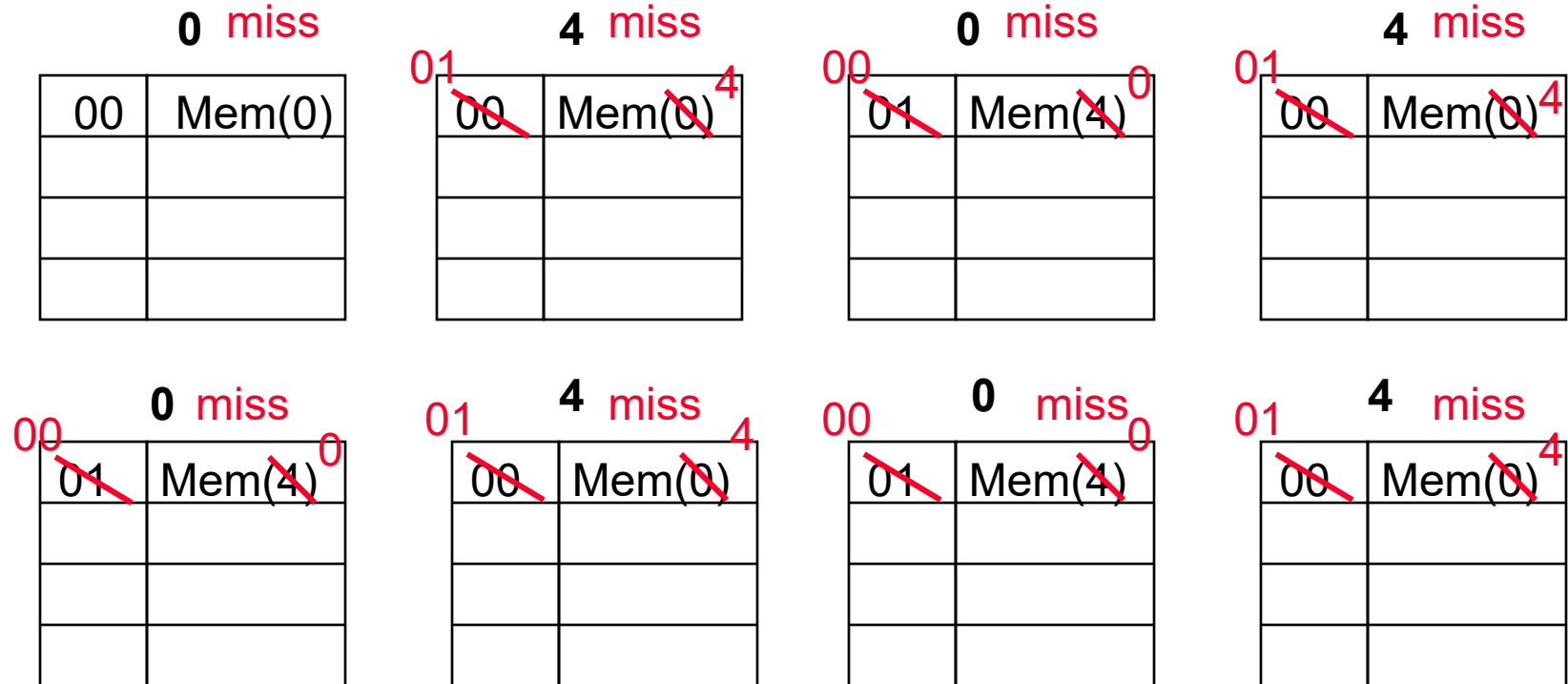
❑ 8 requests, 6 misses

# Another Reference String Mapping

❑ Consider the main memory word reference string

Start with an empty cache - all blocks initially marked as not valid

0 4 0 4 0 4 0 4

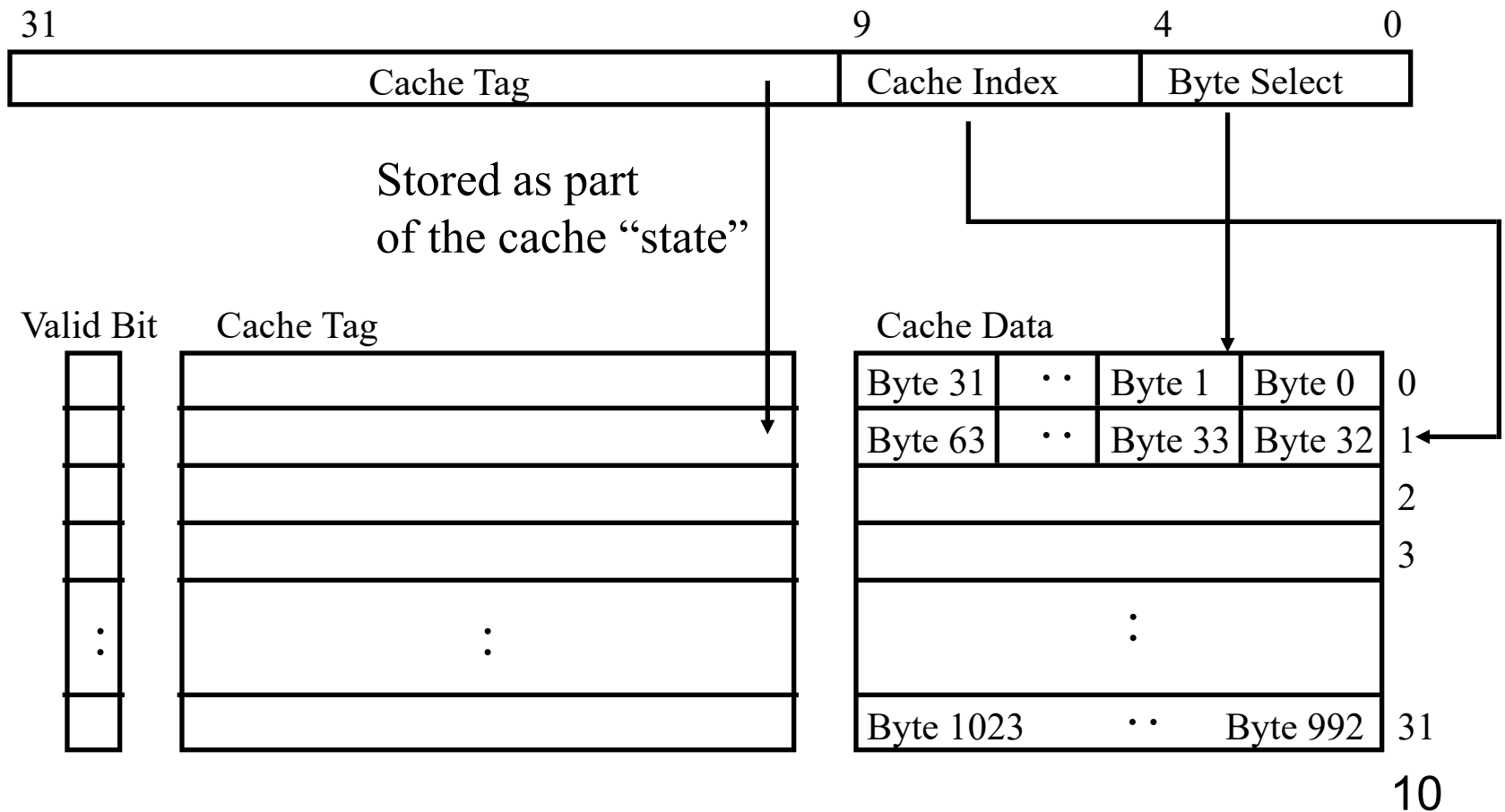


❑ 8 requests, 8 misses

❑ Ping pong effect due to **conflict** misses - two memory locations that map into the same cache block 9

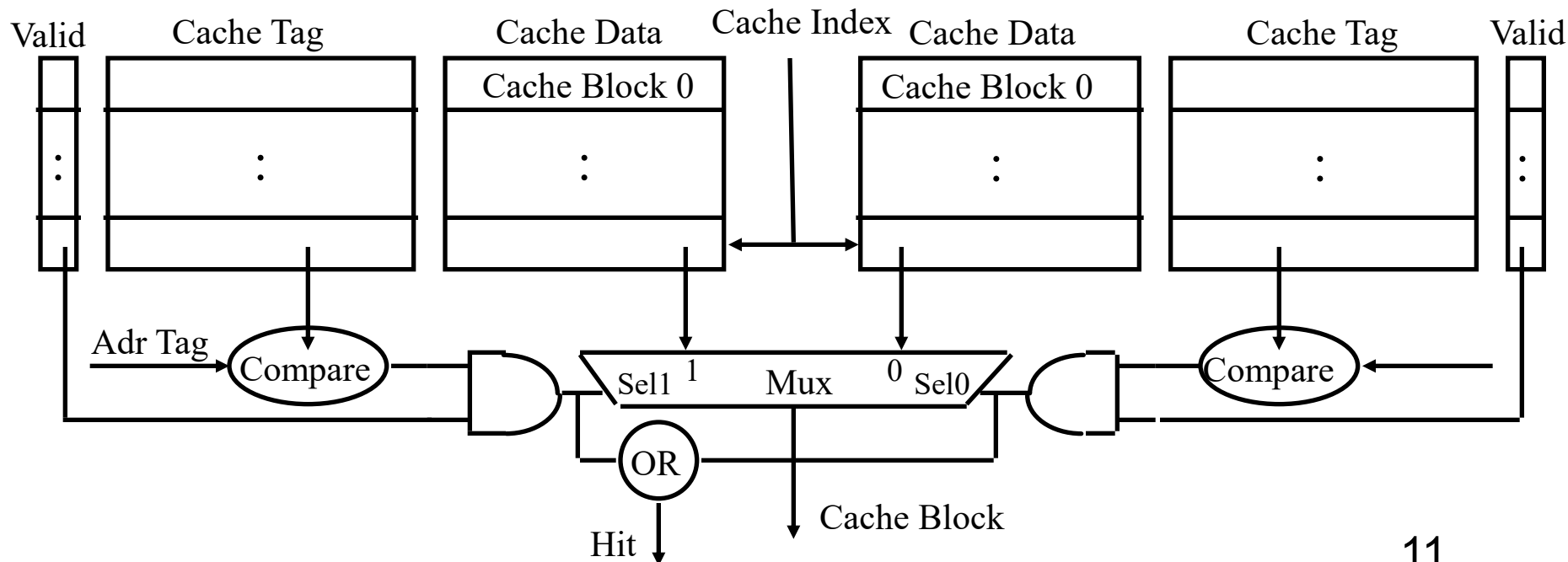


- 10

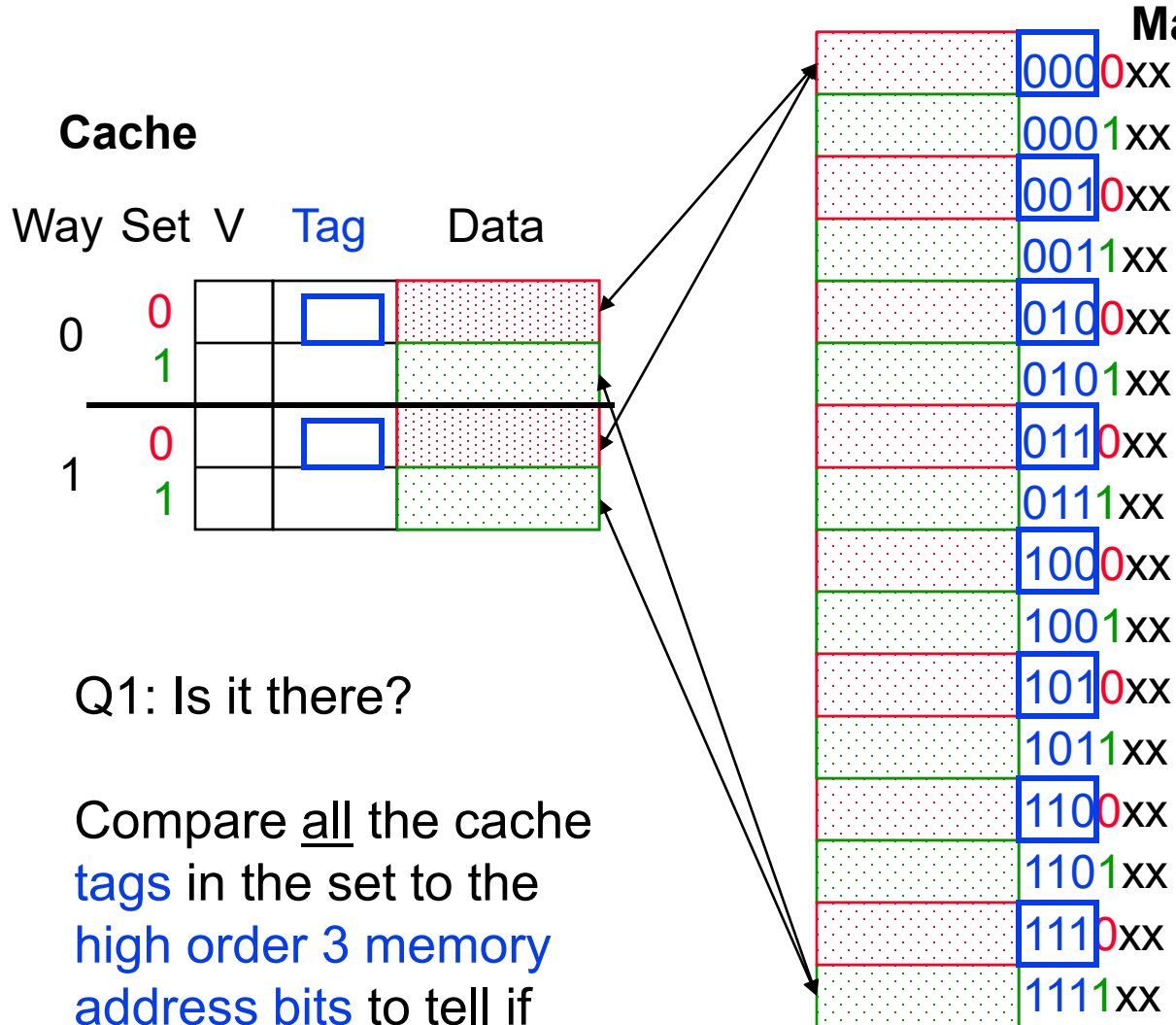


# Set Associative Cache

- N-way set associative:  $N$  entries for each Cache Index
  - $N$  direct mapped caches operates in parallel ( $N$  typically 2 to 8)
- Example: Two-way set associative cache
  - Cache Index selects a “set” from the cache
  - The two tags in the set are compared in parallel
  - Data is selected based on the tag result



# Set Associative Cache Example



One word blocks  
Two low order bits  
define the byte in the  
word (32b words)

Q2: How do we find it?

Use **next 1 low order  
memory address bit** to  
determine which  
cache set (i.e., modulo  
the number of sets in  
the cache)

Q1: Is it there?

Compare all the cache  
**tags** in the set to the  
**high order 3 memory  
address bits** to tell if  
the memory block is in  
the cache

# Another Reference String Mapping

- Consider the main memory word reference string

Start with an empty cache - all blocks initially marked as not valid

0 4 0 4 0 4 0 4

0 miss

|     |        |
|-----|--------|
| 000 | Mem(0) |
|     |        |
|     |        |
|     |        |

4 miss

|     |        |
|-----|--------|
| 000 | Mem(0) |
|     |        |
| 010 | Mem(4) |
|     |        |

0 hit

|     |        |
|-----|--------|
| 000 | Mem(0) |
|     |        |
| 010 | Mem(4) |
|     |        |

4 hit

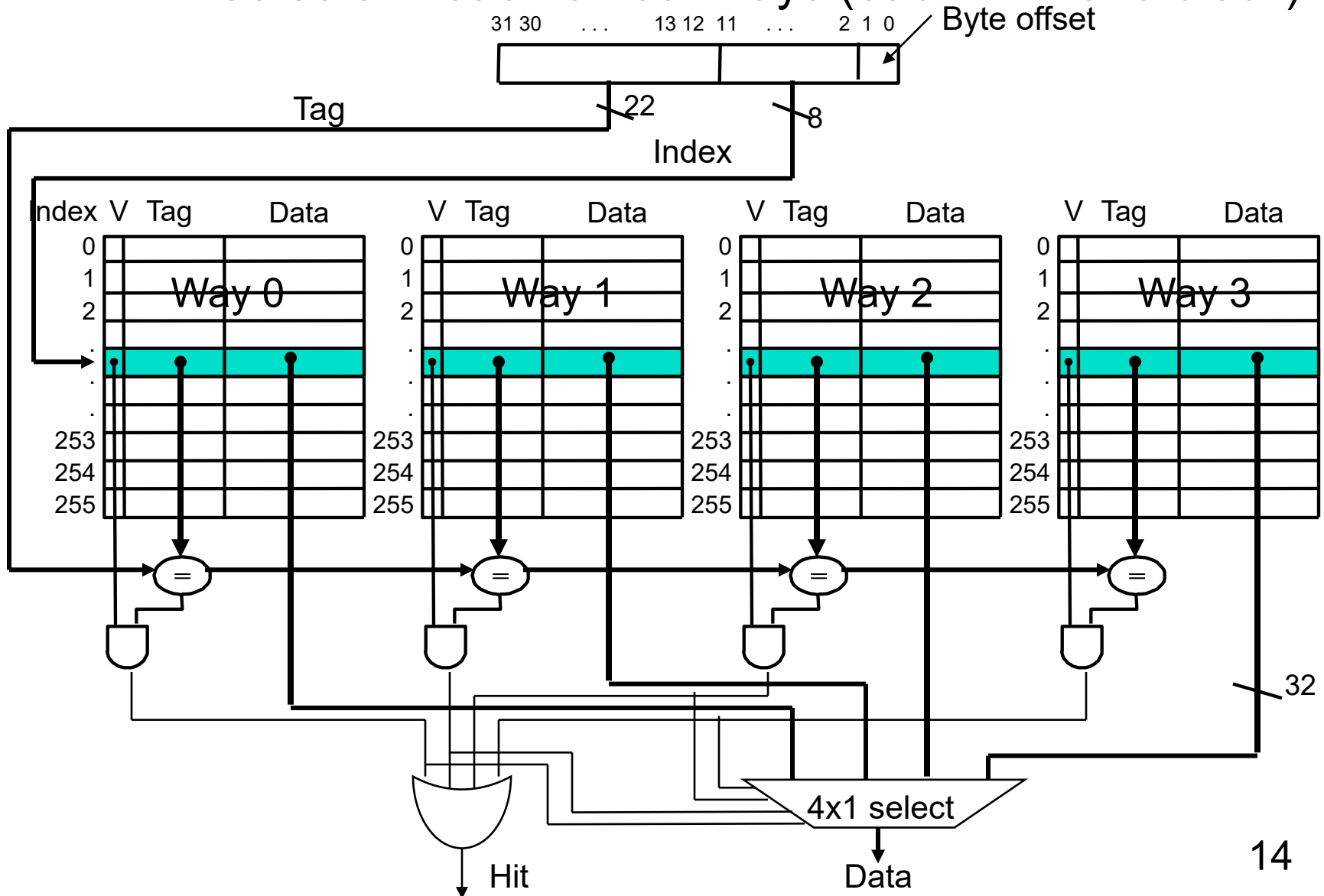
|     |        |
|-----|--------|
| 000 | Mem(0) |
|     |        |
| 010 | Mem(4) |
|     |        |

- 8 requests, 2 misses

- Solves the ping pong effect in a direct mapped cache due to **conflict** misses since now two memory locations that map into the same cache set can co-exist!

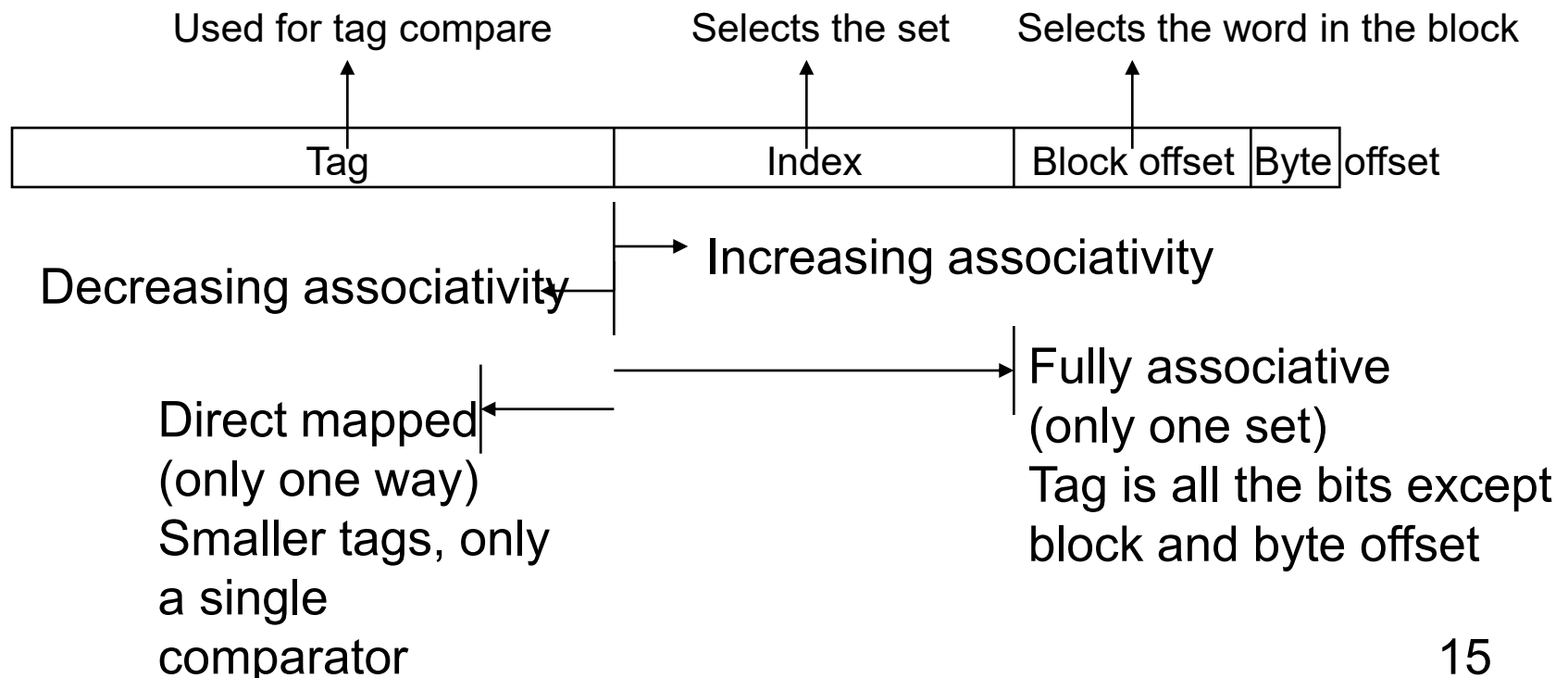
# Four-Way Set Associative 4KB Cache

❑  $2^8 = 256$  sets in each of four ways (each with one block)



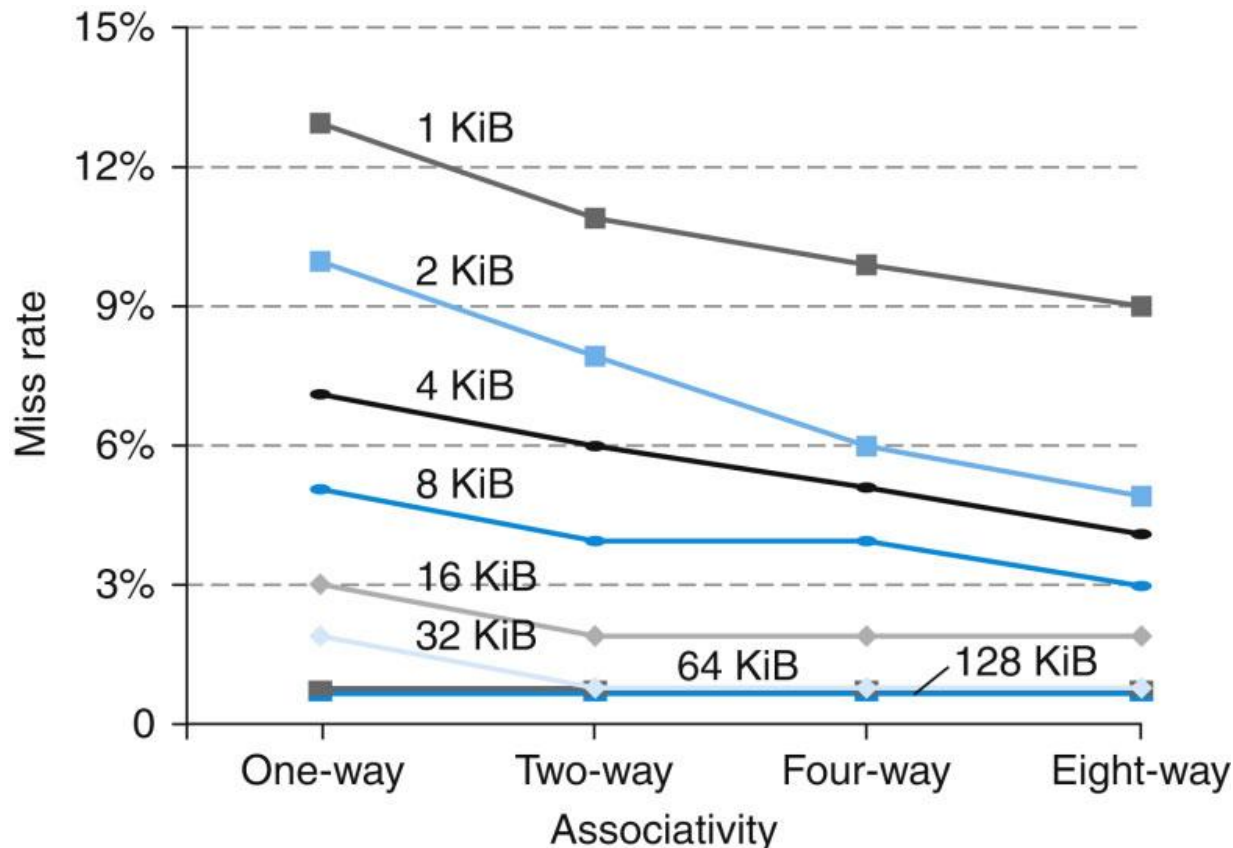
# Range of Set Associative Caches

- For a fixed size cache, each increase by a factor of two in associativity doubles the number of blocks per set (i.e., the number of ways) and halves the number of sets – decreases the size of the index by 1 bit and increases the size of the tag by 1 bit



# Benefits of Set Associative Caches

- ❑ The choice of direct mapped or set associative depends on the cost of a miss versus the cost of implementation



- ❑ Largest gains are in going from direct mapped to 2-way (20%+ reduction in miss rate)

# Costs of Set Associative Caches

- ❑ When a miss occurs, which way's block do we pick for replacement?
  - ❑ **Least Recently Used (LRU)**: the block replaced is the one that has been unused for the longest time
    - Must have hardware to keep track of when each way's block was used relative to the other blocks in the set
    - For 2-way set associative, takes **one bit per way** → set the bit when a block is referenced (and reset the other way's bit)
- ❑ N-way set associative cache additional costs
  - ❑ N comparators (delay and area) – one per way
  - ❑ MUX delay (way selection) before data is available
  - ❑ Data available **after** way selection (and Hit/Miss decision). In a direct mapped cache, the cache block is available **before** the Hit/Miss decision
    - So its not possible to just assume a hit and continue and recover later if it was a miss

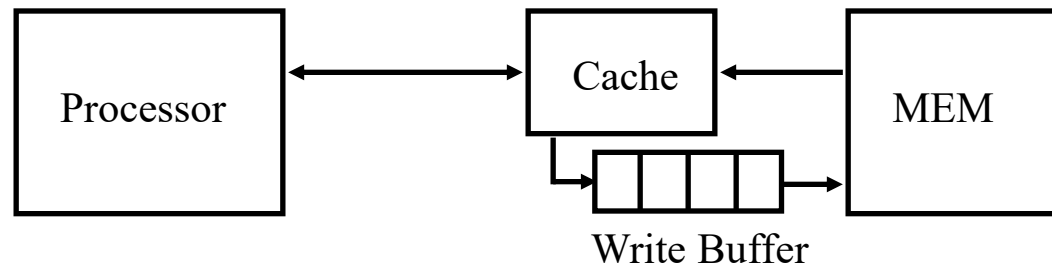
# Block replacement:

- Not an issue for Direct Mapped
- Set Associative or Fully Associative:
  - Random
  - LRU (Least Recently Used)
  - FIFO
  - Belady (ideal but not practical)
  - Others? (ML-based approach)

Designing a Cost-Effective Cache Replacement Policy  
using Machine Learning  
Subhash Sethumurugan, Jieming Yin, John Sartori  
27th IEEE International Symposium on High-Performance  
Computer Architecture (HPCA), 2021.

## Write policies

- Write through or write back,
- Write back can take advantage of a dirty bit
- A Write Buffer (FIFO) is needed between the Cache and Memory
  - Typical number of entries: 4
  - Write buffer saturation??



## Two options on a write miss :

- Write allocate: read block first to cache (as in a read miss), then write the word
- No-write allocate: write only to memory and not to cache

# Average memory access time



Example: Assume that

- Processor speed = 1 GHz (1 n.sec. clock cycle)
- Cache access time = 1 clock cycle
- Miss penalty = 100 n.sec (100 clock cycles)
- I-cache miss rate = 1%, and D-cache miss rate = 3%
- 74% of memory references are for instructions and 26% for data

$$\text{Effective cache miss rate} = 0.01 * 0.74 + 0.03 * 0.26 = 0.0152$$

$$\begin{aligned}\text{Av. (effective) memory access time} &= 1 + 0.0152 * 100 = 2.52 \text{ cycles} \\ &= 2.52 \text{ n.sec}\end{aligned}$$

- Now, assume that you increase the processor speed to 1.5 GHz

$$\text{Miss penalty} = 150 \text{ cycles (0.66 n.sec cycles)}$$

$$\begin{aligned}\text{Av. (effective) memory access time} &= 1 + 0.0152 * 150 = 3.28 \text{ cycles} \\ &= 2.18 \text{ n.sec}\end{aligned}$$

# Cache Performance (§B.2)



$$CPUtime = IC \times (CPI_{execution} + Misses\ per\ instruction \times Miss\ penalty) \times Clock\ cycle\ time$$

$IC$  = Instruction Count

Misses per instruction = Memory accesses per instruction  $\times$  Miss rate

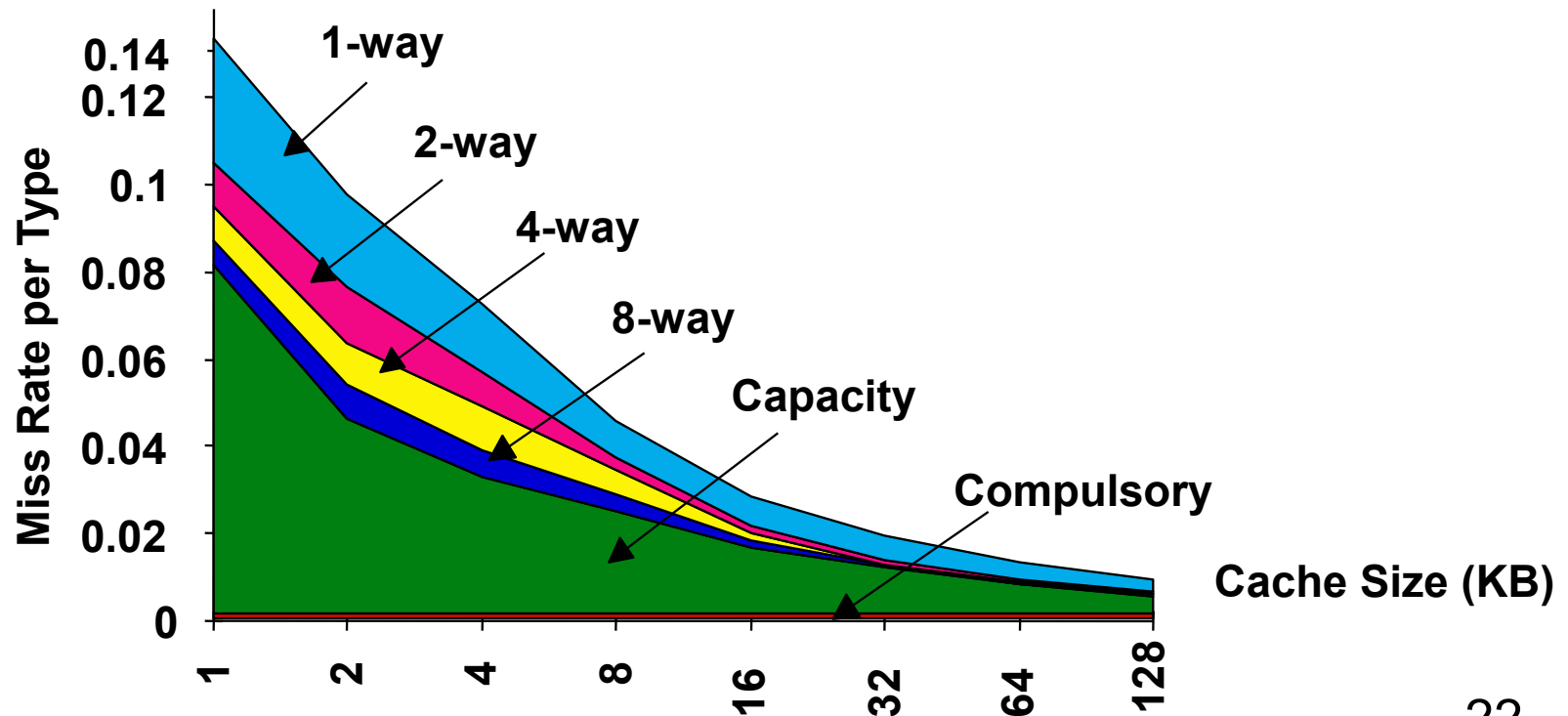
$$CPUtime = IC \times \left( CPI_{Execution} + \frac{Memory\ accesses}{Instruction} \times \mathbf{Miss\ rate} \times Miss\ penalty \right) \times Clock\ cycle\ time$$

## Improving Cache Performance:

1. Reduce the miss rate,
2. Reduce the miss penalty,
3. Reduce the time to hit in the cache.

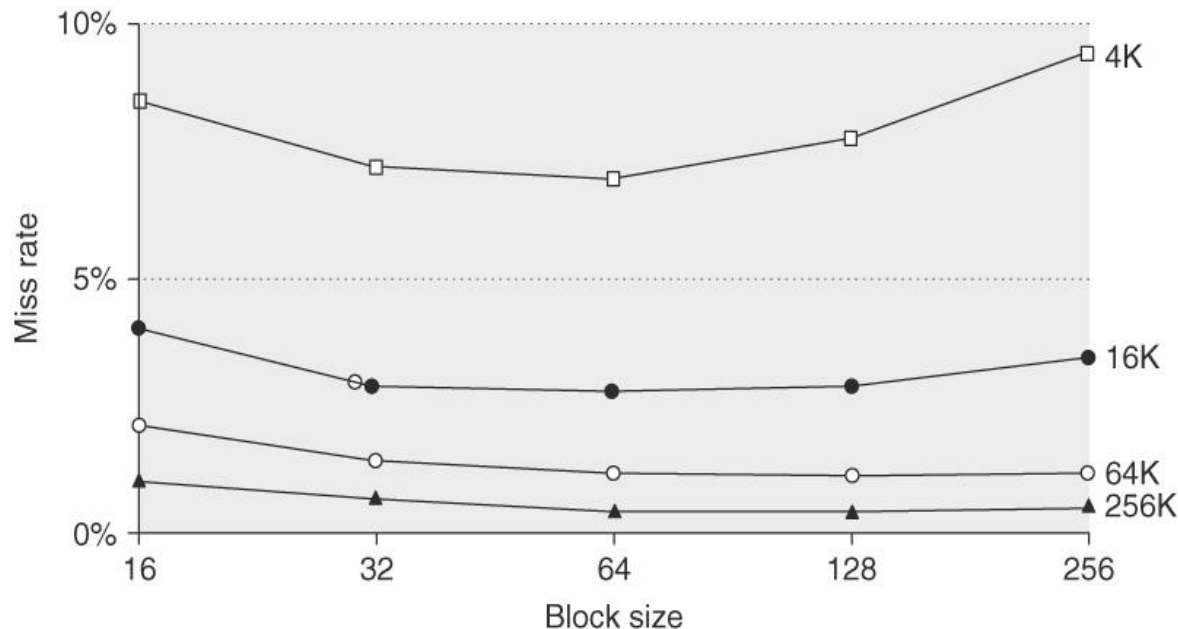
# Classification of cache misses

- Compulsory Misses: Sad facts of life. Example: cold start misses.
- Capacity Misses: Increase cache size
- Conflict Misses: Increase cache size and/or associativity.  
Nightmare Scenario: ping pong effect!



# Basic ways to reduce miss rate (§ B.3)

## 1. Larger Block Size



- Block size explores spatial locality,
- Large blocks are only useful if penalty to fetch a block of K words is less than K times the penalty to fetch one word
- Why can a large block size help and why can it hurt?
- Which of the three C's are affected by the block size?

## 2. Larger caches

- Which of the three C's are affected by the cache size?
- Why can't we increase the size of the cache arbitrarily?

## 3. Higher Associativity

- Which of the three C's are affected by associativity?
- 8-way set associativity is as good as full associativity
- Beware: Execution time is the only final measure!
  - Will hit latency (possibly clock cycle time) increase?
- Effect on power consumption?
- Example: If higher associativity increases the hit time by 20% but decreases the miss rate from 5% to 2.5%, would you go with higher associativity? - Will your decision depend on other factors?

# Basic ways to reduce the Miss Penalty

## 4. Multi-level cache

$$AMAT = Acc\ Time_{L_1} + \text{Miss Rate}_{L_1} \times (Acc\ Time_{L_2} + \text{Miss Rate}_{L_2} \times Miss\ Penalty_{L_2})$$

*Local miss rate*— misses in this cache divided by the total number of memory accesses *to this cache* (Miss rate<sub>L<sub>2</sub></sub>)

*Global miss rate*—misses in this cache divided by the total number of memory accesses *generated by the CPU* (Miss Rate<sub>L<sub>1</sub></sub> x Miss Rate<sub>L<sub>2</sub></sub>)

- L2 is larger (fewer misses) and slower (larger hit time) than L1
- Since hits are few in L2, may target miss penalty reduction
- In L1, may target access time reduction.
- Multilevel inclusion is desirable.
- **Danger:** time to DRAM will grow with multiple levels in between

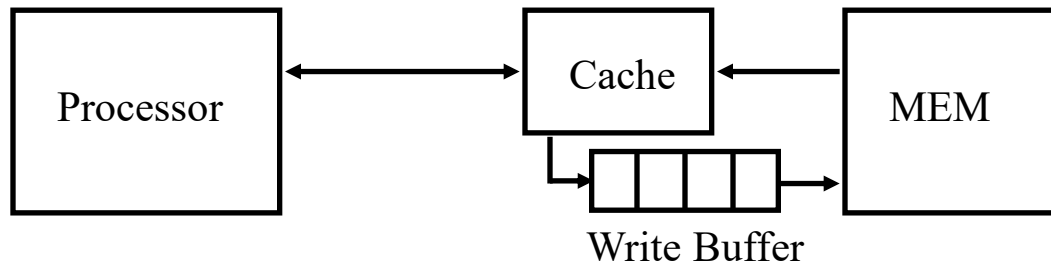
## Example

- Miss rate of L1 = 4%, acc time is one cycle
- If L2 is direct mapped, local miss rate = 25% and acc time = 10 cycles
- If L2 is set-associative, local miss rate = 20% and acc time = 11 cycles
- Miss penalty for L2 is 100 cycle.

AMAT with direct mapped L2 =  $1 + 0.04 (10 + 0.25 * 100) = 2.4$  cycles

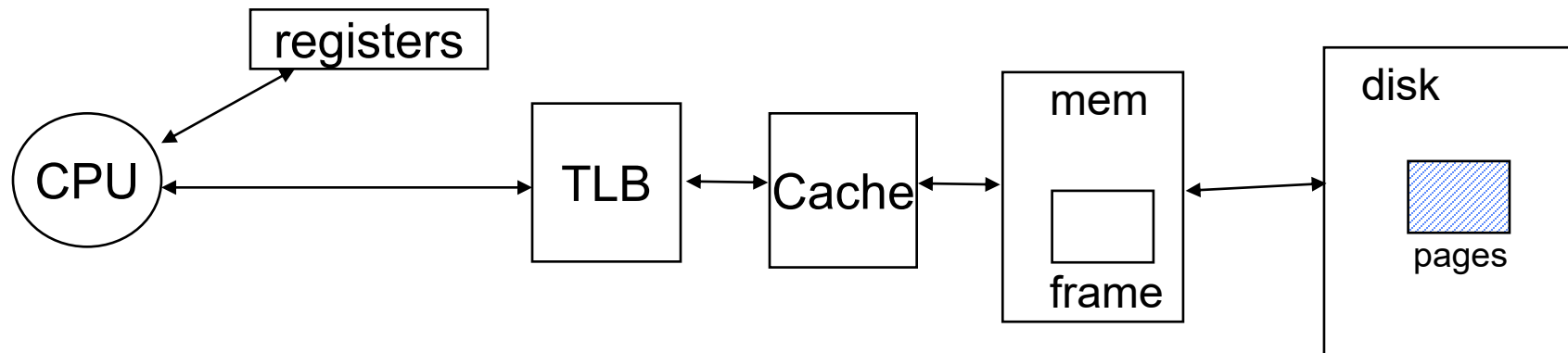
AMAT with set-associative L2 =  $1 + 0.04 (11 + 0.20 * 100) = 2.24$  cycles

## 5. Read Priority over Write on Miss

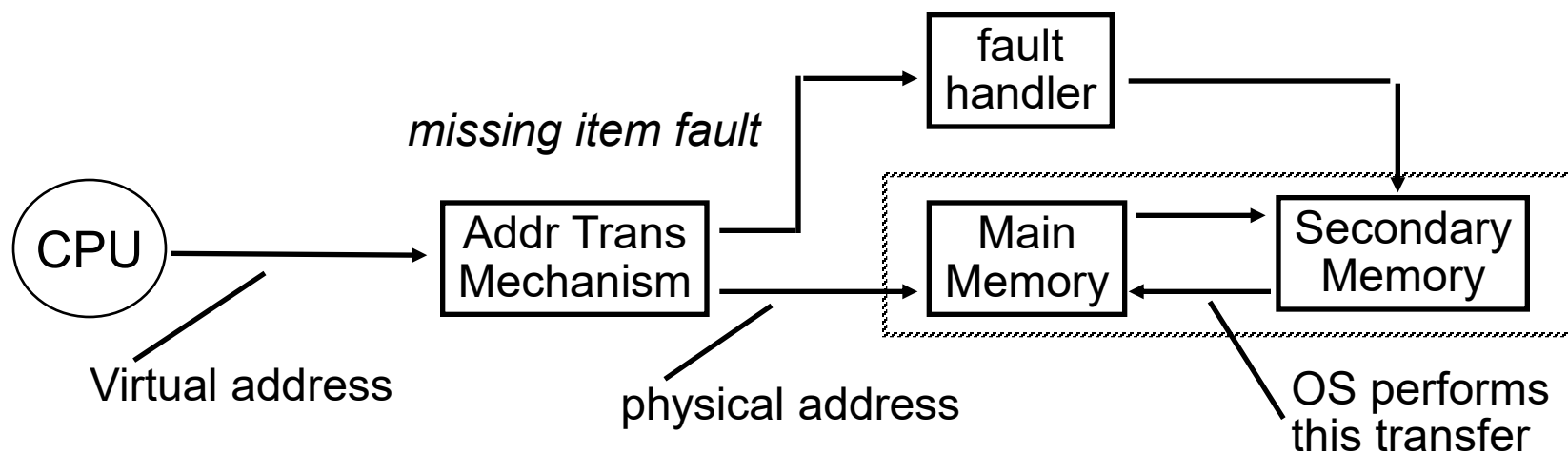


- If write through, write buffers avoid stalling
- May causes RAW conflicts with reads on cache misses
- Waiting for write buffer to empty will increase read miss penalty
  - Solution: Check write buffer contents before read;  
if no conflicts, let the memory access continue
- If write Back (replace dirty block on a miss)
  - CPU stalls less since restarts as soon as the read completes
  - Cache coherence problem in multi-core execution. Stay tuned.

# Basic Issues in VM System Design (§ B.4)



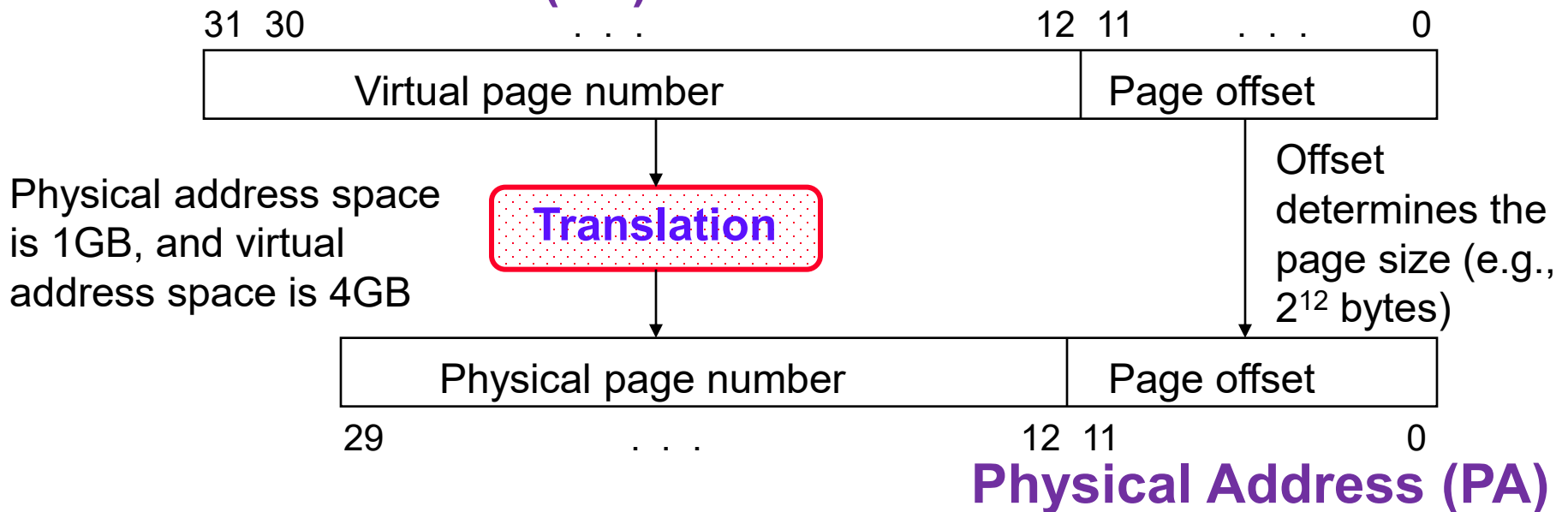
missing item fetched from secondary memory only on the occurrence of a fault --> *load on demand policy*



# Address Translation

- ❑ A **virtual address** is translated to a **physical address** by a combination of hardware and software

## Virtual Address (VA)



- ❑ So, each memory request *first* requires an **address translation** from the virtual space to the physical space

# Virtual Address Translation Mechanisms

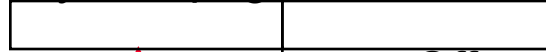
Virtual page #    Offset



Virtual Address

The page table together with the program counter and the registers specifies the **state** of a program

Physical page #



Physical Address

Offset

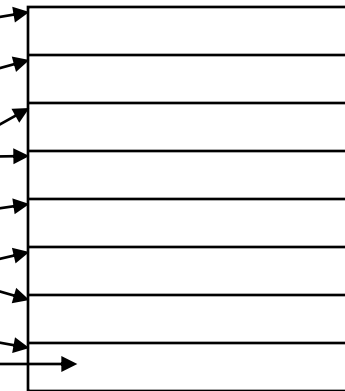
Page Table Register

Physical page base addr

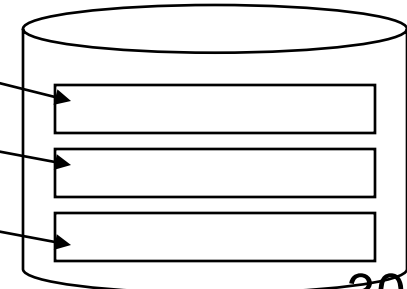
| V |   |  |
|---|---|--|
| 1 | • |  |
| 1 | • |  |
| 1 | • |  |
| 1 | • |  |
| 1 | • |  |
| 1 | • |  |
| 1 | • |  |
| 0 | • |  |
| 1 | • |  |
| 0 | • |  |
| 1 | • |  |
| 0 | • |  |

Page Table

(stored in main memory)



Main memory



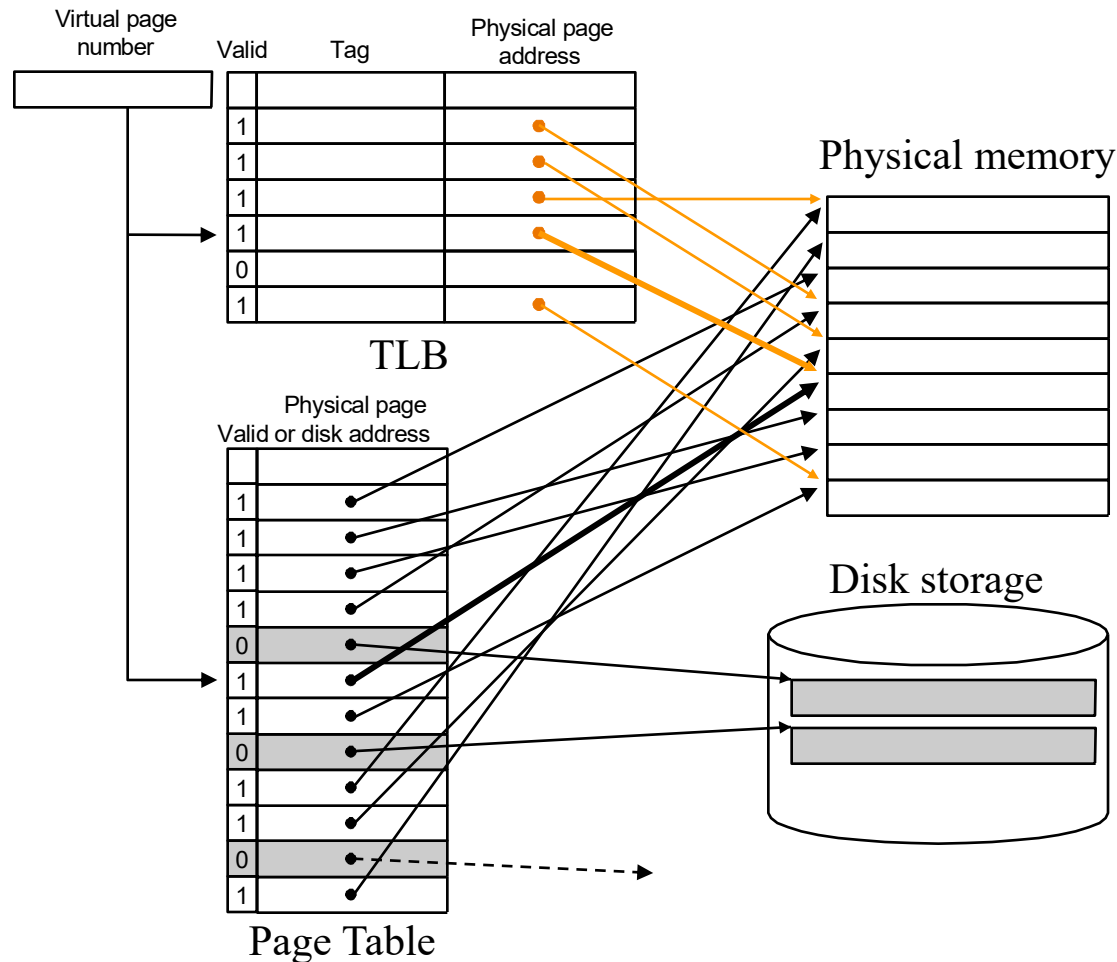
Disk storage

30

If **valid bit** for a virtual page is off (0), a page fault occurs

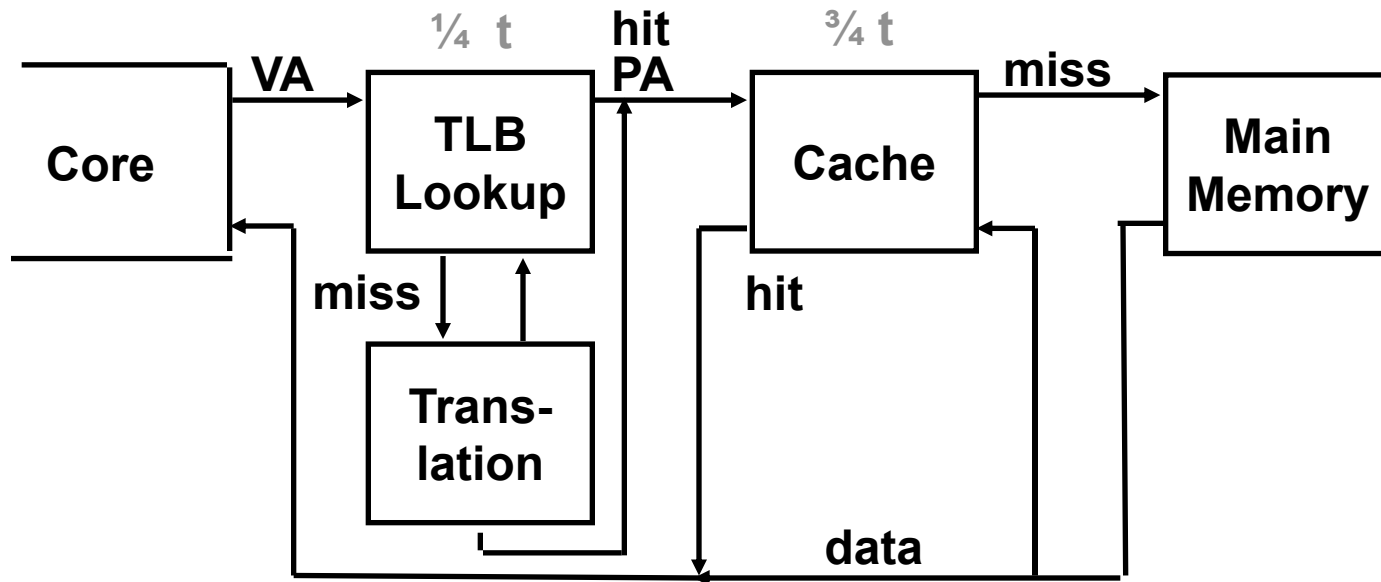
# Making Address Translation Fast

Cache the currently used entries of PT in a *Translation Lookaside Buffer* (TLB).



- What if we get a TLB miss (page table entry is not in TLB)?

# A TLB in the Memory Hierarchy



- ❑ **A TLB miss – is it a page fault or merely a TLB miss?**
  - ❑ If the page is loaded into main memory, then the TLB miss can be handled (in hardware or software) by loading the translation information from the page table into the TLB
    - Takes 100's of cycles to find and load the translation info into the TLB
  - ❑ If the page is not in main memory, then it's a *true* page fault
    - Takes 1,000,000's of cycles to service a page fault
- ❑ TLB misses are much more frequent than true page faults

# Handling a TLB Miss

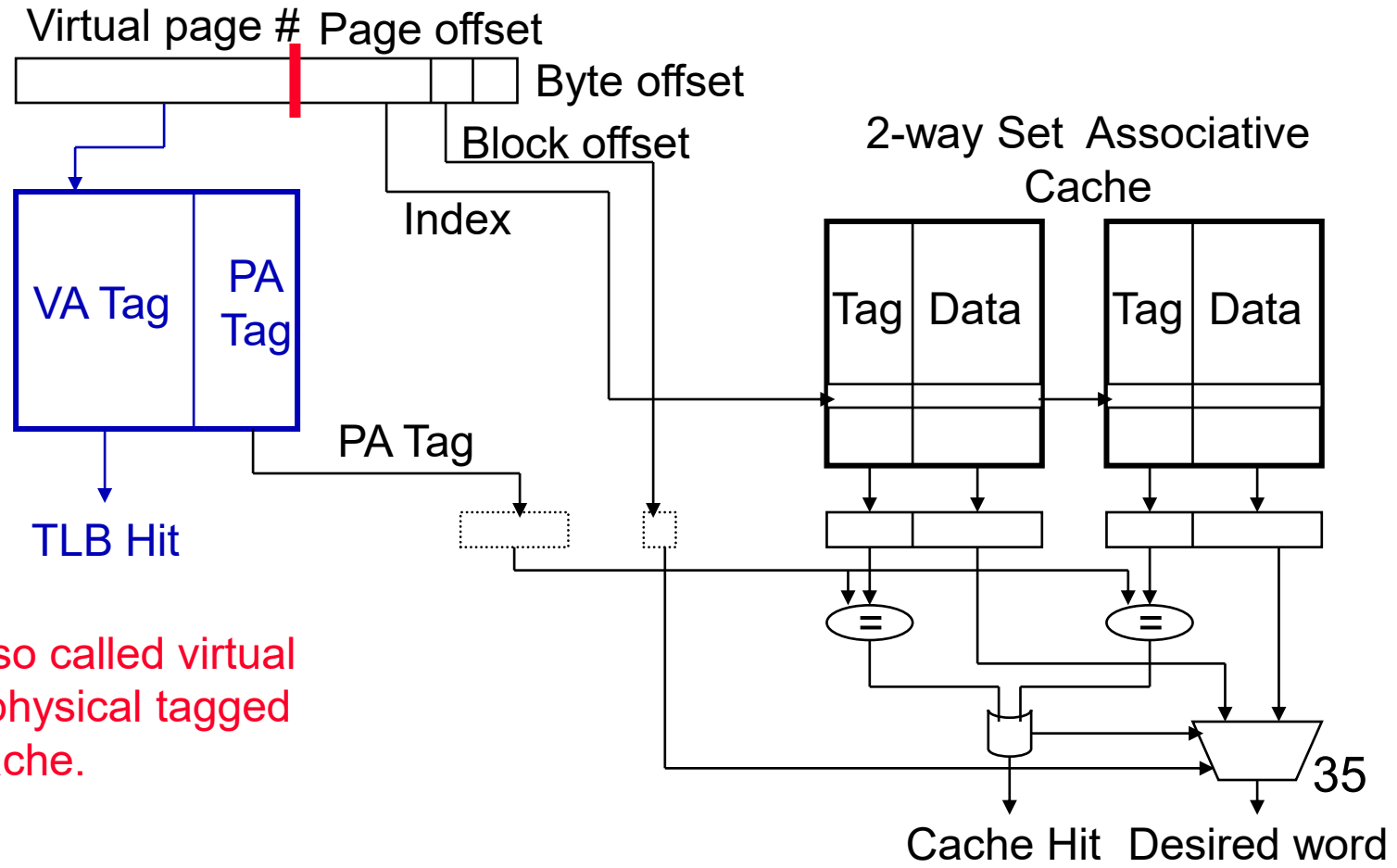
- ❑ A **TLB miss** can indicate one of two possibilities:
  - ❑ A page is present in memory, and we need only create the missing TLB entry
  - ❑ A page is not present in memory, and we need to transfer control to the operating system to deal with a page fault

# Handling a TLB Miss

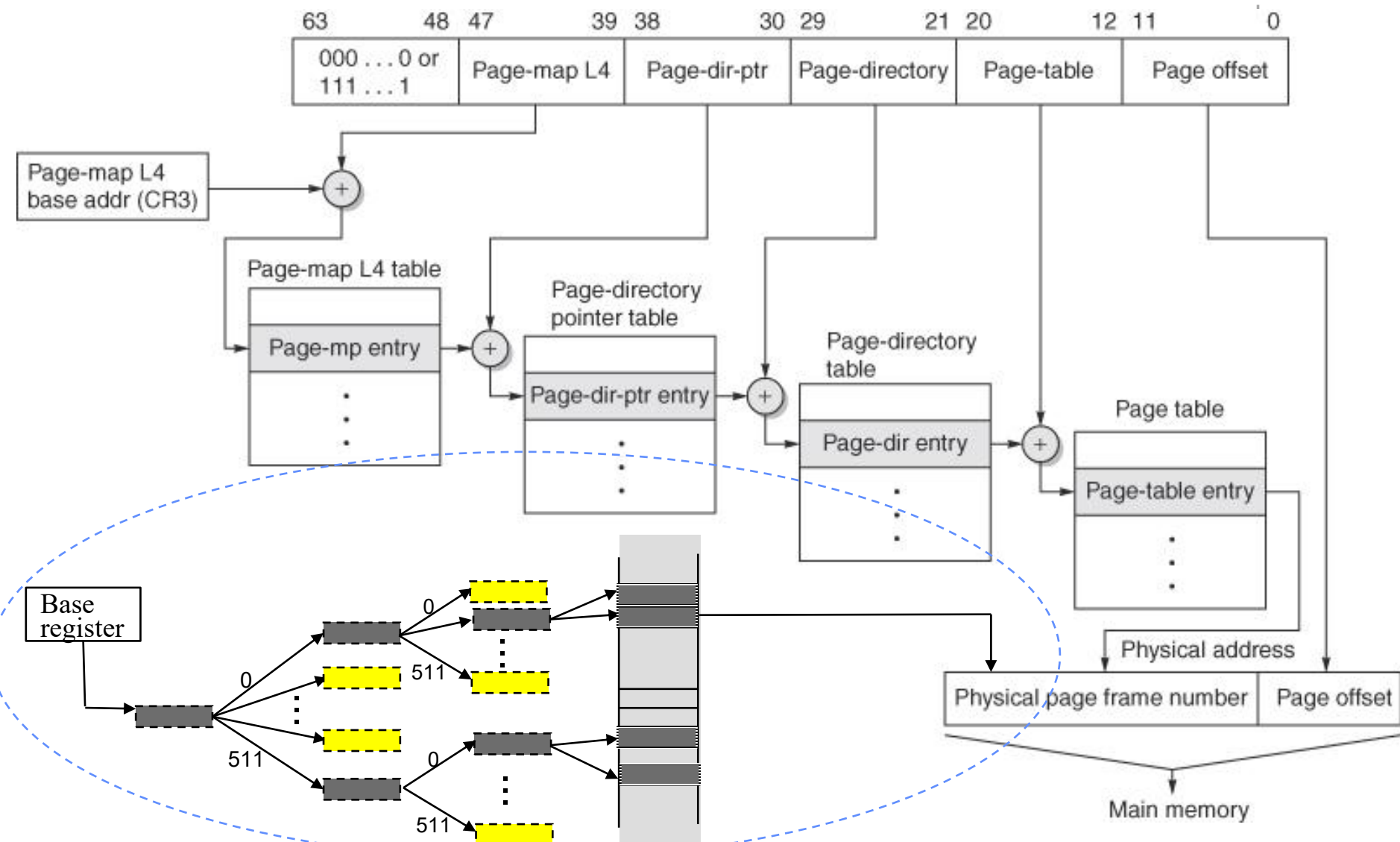
- ❑ Once the operating system knows the virtual address that caused the page fault, it must complete three steps:
  - ❑ Look up the page table entry using the virtual address and find the location of the referenced page on disk
  - ❑ Choose a physical page to replace; if the chosen page is dirty, it must be written out to disk before we can bring a new virtual page into this physical page
  - ❑ Start a read to bring the referenced page from disk into the chosen physical page
- ❑ The last step will take *millions of clock cycles* (so will the second if the replaced page is dirty)
- ❑ Accordingly, the operating system will usually select/schedule another process to execute in the processor *until* the disk access completes
- ❑ When the read of the page from the disk completes, the operating system can restore the state of the process that originally caused the page fault and execute the instruction that returns from the exception
- ❑ The user process (application) then re-executes the instruction that faulted

# Further Reducing Translation Time

- ❑ Can **overlap** the cache access with the TLB access
- ❑ Overlapped access only works as long as the address bits used to index into the cache **do not change** as the result of VA translation
- ❑ This usually limits things to small caches, large page sizes, or high n-way set associative caches if you want a large cache



This is also called virtual indexed physical tagged (VIPT) cache.



**Figure B.27 The mapping of an Opteron virtual address.** The Opteron virtual memory implementation with four page table levels supports an effective physical address size of 40 bits. Each page table has 512 entries, so each level field is 9 bits wide. The AMD64 architecture document allows the virtual address size to grow from the current 48 bits to 64 bits, and the physical address size to grow from the current 40 bits to 52 bits.

# The whole picture

